Détection DDoS et généralisation à des attaques non observées

Projet DAF — détection d'anomalies et fraude avec Python

2026

Sommaire

Résumé 1

1. Question de recherche et hypothèses 2

2. Données et analyse exploratoire 2

3. Protocole expérimental 4

4. Expérience 1 — Binaire vs multiclasses (H1, Q2) 4

5. Expérience 2 — Leave-One-Attack-Out (H2, Q1) 6

6. Expérience 3 — Réduction de features (H3, Q3, Q4) 8

7. Explicabilité et analyse d'erreurs 10

8. Discussion, limites et recommandations 11

9. Utilisation de l'IA 12

10. Conclusion 12

Annexe A — Commandes de reproduction 12

Annexe B — Fichiers de résultats 12

Résumé

Ce projet étudie la généralisation d'un détecteur DDoS entraîné sur certaines familles d'attaques à une famille jamais observée. Le jeu de données CIC-DDoS2019 contient 431 371 flux bruts, 77 features et 17 familles fines. Deux artefacts doivent être neutralisés avant toute modélisation : 21 347 lignes appartiennent à des groupes de doublons exacts, et le split publié `-training` / `-testing` partitionne les mécanismes d'attaque au lieu de tirer au hasard. Après dédoublonnage, 418 884 lignes restent et un split stratifié 80/20 est créé une fois par graine.

La classification binaire Benign/DDoS est facile (Macro-F1 = 0.9991), mais la classification en 13 classes tombe à 0.7566 pour la forêt aléatoire. En Leave-One-Attack-Out, dix familles restent au-dessus de 0,96 (neuf au-dessus de 0,99), mais **Syn s'effondre à 0.6035** avec la forêt aléatoire,

alors que la régression logistique atteint 0.9733 sur la même famille. La dépendance à la famille est donc réelle, mais concentrée et dépendante du modèle. La réduction de features conserve l'essentiel de la performance, mais le gain de latence d'une forêt aléatoire reste modeste.

1. Question de recherche et hypothèses

1.1 Question centrale

> Un modèle entraîné sur certaines attaques DDoS reconnaît-il une stratégie DDoS jamais rencontrée ?

Les questions d'analyse imposées sont :

- **Q1** — Quelles familles DDoS généralisent le mieux vers d'autres familles ?
- **Q2** — Un modèle très performant en binaire est-il nécessairement utile pour identifier le type d'attaque ?
- **Q3** — Combien de variables sont réellement nécessaires ?
- **Q4** — Quels résultats sont les plus importants pour une utilisation en temps réel ?

1.2 Hypothèses

- **H1** — La classification multiclasses est plus difficile que la classification binaire.
- **H2** — La capacité de généralisation dépend fortement de la famille DDoS laissée de côté.
- **H3** — Une fraction des variables conserve une grande partie des performances, avec un coût d'inférence réduit.

2. Données et analyse exploratoire

2.1 Composition

Le dataset est composé de 17 fichiers Parquet, 431 371 lignes et 77 features numériques. Les labels bruts sont ramenés à 17 familles fines : `Benign` et 16 familles d'attaque, en conservant `DrDoS_*` séparé de sa contrepartie d'exploitation. Le libellé `UDP-lag` est normalisé en `UDPLag`.

Famille	Lignes	Part (%)
DrDoS_NTP	121 368	28.14
TFTP	98 917	22.93
Benign	97 831	22.68
Syn	49 373	11.45
UDP	18 090	4.19
DrDoS_UDP	10 420	2.42
UDPLag	8 927	2.07
MSSQL	8 523	1.98
DrDoS_MSSQL	6 212	1.44
DrDoS_DNS	3 669	0.85
DrDoS_SNMP	2 717	0.63
LDAP	1 906	0.44
DrDoS_LDAP	1 440	0.33
Portmap	685	0.16
NetBIOS	644	0.15
DrDoS_NetBIOS	598	0.14
WebDDoS	51	0.01

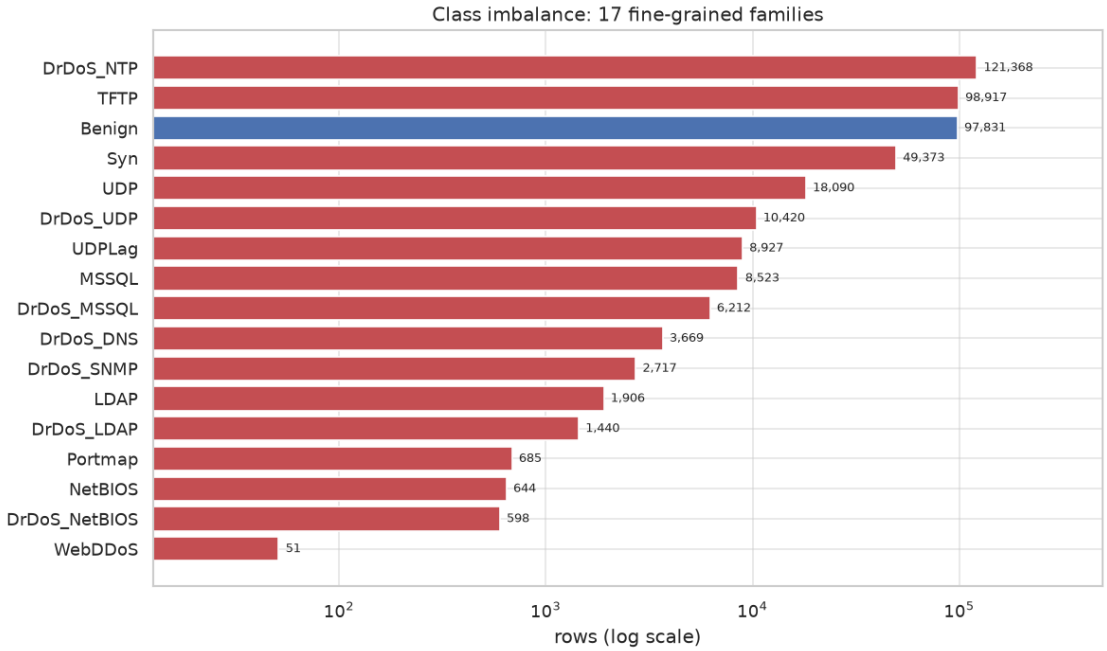


Figure 1: Déséquilibre des familles

Le déséquilibre est extrême : `DrDoS_NTP`, `TFTP` et `Benign` représentent environ 74 % des lignes, tandis que `WebDDoS` ne compte que 51 lignes. Ce déséquilibre justifie Macro-F1 et le rappel par classe plutôt que l'accuracy.

2.2 Protocole et familles

La crosstabulation famille × `Protocol` montre que `Syn` est TCP à 99,8 %, `MSSQL` et `UDP` sont UDP à plus de 99,9 %. `Protocol` est donc un raccourci potentiel pour le multiclasses ; l'expérience 1 le retire dans une ablation.

Le split publié est organisé par mécanisme. Les sept familles `DrDoS_*` n'apparaissent que dans les fichiers `-testing`, alors que `LDAP`, `MSSQL`, `NetBIOS`, `Portmap` et `UDP` n'apparaissent que dans les fichiers `-training`. Seules `Benign`, `Syn` et `UDPLag` chevauchent les deux. Utiliser ce split mesurerait la reconnaissance d'un mécanisme, pas la généralisation. Il est donc écarté.

2.3 Audit des fuites

Contrôle	Valeur
Lignes dans des groupes de doublons exacts	21 347
Empreintes présentes dans plus d'un fichier source	8 858
Empreintes avec labels contradictoires	5 289
Lignes impliquées dans ces conflits	12 957
Lignes retirées par dédoublonnage	12 487
Lignes après dédoublonnage	418 884

- 21 347 lignes sont dans des groupes de doublons exacts.
- 8 858 empreintes de features apparaissent dans plus d'un fichier source.

- 5 289 empreintes ont des labels contradictoires (12 957 lignes). Certaines attaques par réflexion et leur contrepartie d'exploitation partagent le même vecteur de features ; c'est une limite intrinsèque du dataset.
- Le dédoublonnage retire 12 487 lignes et laisse 418 884 lignes.

La règle retenue est : une ligne par empreinte de features, label majoritaire en cas de conflit, égalité tranchée lexicographiquement. Elle est déterministe et ne dépend pas de l'ordre des fichiers.

3. Protocole expérimental

3.1 Split et pipelines

Un split stratifié 80/20 est créé sur la famille fine, une fois par graine, et réutilisé par toutes les expériences. La cible binaire est dérivée des mêmes lignes, ce qui garantit que les comparaisons binaire/multiclasses portent sur exactement le même test. Les graines sont 42 pour la baseline et E1, puis 42/43/44 pour E2 et E3.

Tout preprocessing vit dans une `Pipeline` ajustée sur le train :

1. suppression des colonnes constantes (12 colonnes dans ce dataset) ;
2. standardisation (`StandardScaler`) ;
3. éventuellement sélection de features (`SelectKBest` + `mutual_info_classif`) ;
4. classifieur.

Aucune statistique de test n'entre dans le fit. La sélection de features de E3 est donc fitted on train fold only.

3.2 Modèles

- `DummyClassifier(strategy="stratified")` : plancher.
- `LogisticRegression(max_iter=2000, class_weight="balanced")` : référence linéaire.
- `RandomForestClassifier(n_estimators=300, class_weight="balanced")` : référence non linéaire.

3.3 Métriques

Macro-F1, Weighted-F1, précision/rappel/F1 par classe, PR-AUC, ROC-AUC et FPR à TPR = 95 %. L'accuracy est affichée mais n'est jamais utilisée pour conclure. Le FPR est le taux de Benign classés DDoS ; c'est la métrique opérationnelle la plus lisible pour un SOC.

4. Expérience 1 — Binaire vs multiclasses (H1, Q2)

4.1 Protocole

Mêmes lignes, même split, mêmes modèles, même graine. Seule la cible change : Benign/DDoS, puis 13 classes (Benign + 12 types d'attaque obtenus en fusionnant `DrDoS_X` et `X`). Une troisième variante retire `Protocol`.

4.2 Résultats

Tâche	Modèle	Macro-F1	Weighted-F1	Accuracy	PR-AUC	ROC-AUC
Binaire	Dummy	0.5006	0.6507	0.6508	0.7741	0.5006
Binaire	LogReg	0.9949	0.9964	0.9964	0.9998	0.9994
Binaire	RF	0.9991	0.9994	0.9994	1.0000	1.0000
13 classes	Dummy	0.0763	0.2080	0.2082	0.0768	0.4996
13 classes	LogReg	0.6029	0.9440	0.9310	0.5932	0.9887
13 classes	RF	0.7566	0.9816	0.9815	0.7786	0.9945
Sans Protocol	Dummy	0.0763	0.2080	0.2082	0.0768	0.4996
Sans Protocol	LogReg	0.6007	0.9439	0.9308	0.5920	0.9886
Sans Protocol	RF	0.7577	0.9817	0.9816	0.7790	0.9948



Figure 2: Macro-F1 et Weighted-F1

La forêt aléatoire passe de 0.9991 en binaire à 0.7566 en 13 classes. Le Weighted-F1 reste à 0.9816 : les familles majoritaires sont bien classées, mais les familles rares tirent le Macro-F1 vers le bas. C'est exactement le piège que l'accuracy seule masquerait.

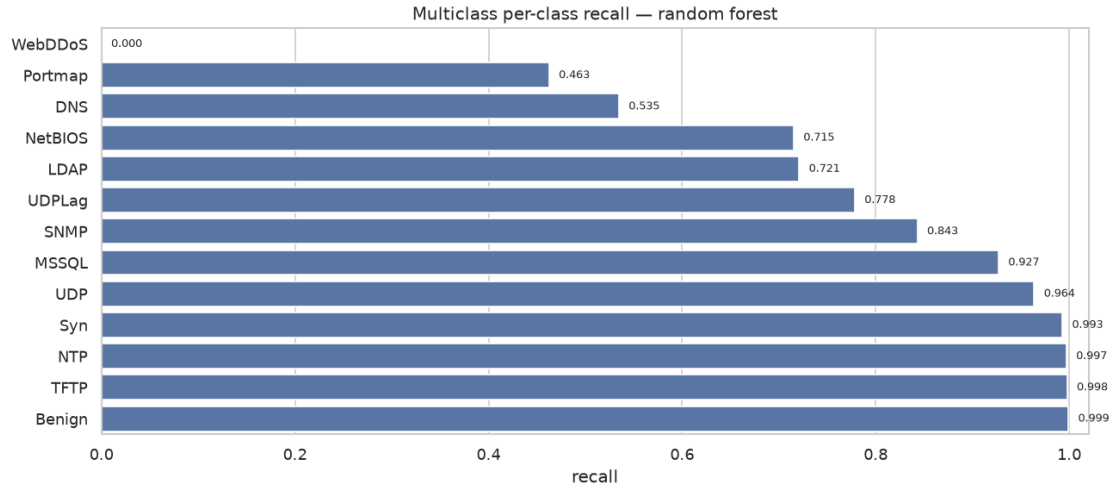


Figure 3: Rappel par classe

L'ablation de `Protocol` fait passer la Macro-F1 de 0.7566 à 0.7577, soit un écart de 0.0011. Le

multiclasses ne repose donc pas seulement sur une lecture du protocole : les champs d'en-tête TCP (longueur d'en-tête, fenêtres initiales) et le débit portent l'essentiel du signal.

4.3 Verdict

H1 est soutenue. Un modèle presque parfait en binaire perd beaucoup en multiclasses, surtout en Macro-F1. **Q2 : non**, un excellent détecteur binaire n'est pas nécessairement utile pour identifier le type d'attaque. Un alarme binaire est une porte de triage ; elle ne dit pas quel playbook déclencher.

Interprétation cyber. La confusion entre familles proches est le vrai risque opérationnel : l'analyste reçoit une alerte mais peut appliquer la mauvaise contre-mesure. *Limite.* La fusion `DrDoS_X/X` masque la séparabilité des variantes ; c'est l'objet de E2.

5. Expérience 2 — Leave-One-Attack-Out (H2, Q1)

5.1 Protocole

On tient à l'écart chacune des 11 familles fines ayant au moins 1 000 lignes après dédoublonnage. La famille est purgée des 17 fichiers concaténés, pas seulement du fichier qui porte son nom. Les familles exclues par le seuil de support sont `DrDoS_LDAP` (993), `Portmap` (400), `NetBIOS` (374), `DrDoS_NetBIOS` (311) et `WebDDoS` (49). Ce sont des contraintes d'estimation, pas un jugement sur leur importance.

Le test est équilibré : toutes les lignes disponibles de la famille tenue à l'écart, plus un échantillon apparié de Benign issu uniquement du test des familles connues. Trois graines (42, 43, 44) donnent $11 \times 3 \times 3 = 99$ fits.

L'audit montre qu'une famille peut apparaître dans plusieurs fichiers : `Syn` est présent dans `Syn-training`, `Syn-testing` et `UDPLag-training` ; `UDP` dans `UDP-training` et `UDPLag-training` ; `LDAP` dans `LDAP-training` et `MSSQL-training` ; `NetBIOS` dans `NetBIOS-training` et `LDAP-training`. Purger uniquement le fichier homonyme aurait laissé des lignes de la famille en entraînement.

5.2 Résultats

Famille tenue à l'écart	RF Macro-F1	LR Macro-F1	RF rappel	LR rappel	RF FPR
DrDoS_MSSQL	0.9995	0.9980	0.9997	0.9993	0.0006
DrDoS_SNMP	0.9995	0.9979	1.0000	0.9983	0.0009
DrDoS_UDP	0.9995	0.9978	0.9999	0.9994	0.0008
MSSQL	0.9995	0.9981	0.9998	0.9997	0.0008
UDP	0.9993	0.9980	0.9999	0.9999	0.0013
TFTP	0.9992	0.9973	0.9997	0.9989	0.0012
DrDoS_NTP	0.9988	0.9808	0.9984	0.9652	0.0009
LDAP	0.9985	0.9963	0.9981	0.9952	0.0010
UDPLag	0.9956	0.9934	0.9922	0.9912	0.0010
DrDoS_DNS	0.9699	0.9664	0.9404	0.9361	0.0006
Syn	0.6035	0.9733	0.3041	0.9499	0.0010

Dix familles sur onze restent au-dessus de 0,96 (neuf au-dessus de 0,99), par exemple **DrDoS_MSSQL** : 0.9995, rappel 0.9997, FPR 0.0006. La onzième, **Syn**, s'effondre à 0.6035 avec la forêt aléatoire (rappel 0.3041, FPR 0.0010), alors que la régression logistique atteint 0.9733 sur la même famille. Le FPR reste bas : c'est le **rappel** qui s'effondre, donc des attaques Syn ne

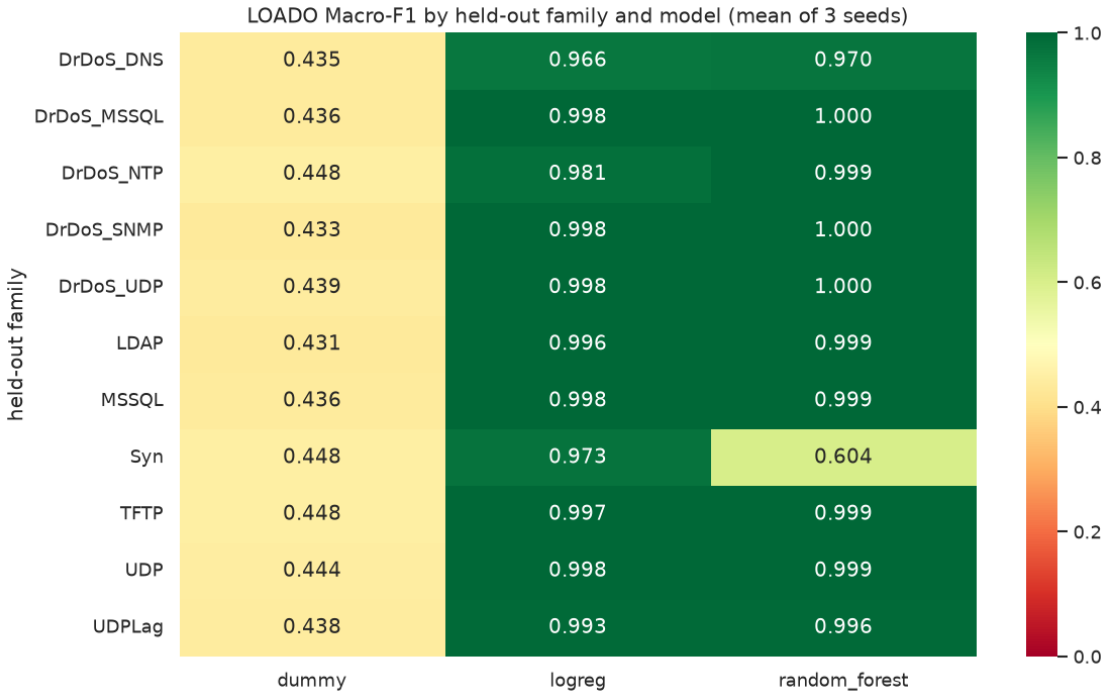


Figure 4: Macro-F1 par famille tenue à l'écart

sont pas vues. L'écart est donc concentré sur une famille et dépend du modèle, pas seulement de la famille.

5.3 Matrice de confusion vers les familles connues

Pour chaque famille inconnue, un modèle multiclasses plus léger, entraîné sur les familles restantes, prédit la famille connue la plus proche.

Famille inconnue	Famille(s) connue(s) la plus proche
DrDoS_DNS	LDAP (0.25), DrDoS_LDAP (0.24), DrDoS_MSSQL (0.13)
DrDoS_MSSQL	MSSQL (0.83), DrDoS_DNS (0.12), DrDoS_SNMP (0.02)
DrDoS_NTP	DrDoS_DNS (0.66), UDP (0.18), DrDoS_UDP (0.13)
DrDoS_SNMP	DrDoS_DNS (0.35), DrDoS_LDAP (0.31), LDAP (0.18)
DrDoS_UDP	UDP (0.95), UDPLag (0.03), DrDoS_DNS (0.01)
LDAP	DrDoS_LDAP (0.60), DrDoS_DNS (0.26), DrDoS_SNMP (0.11)
MSSQL	DrDoS_MSSQL (0.88), DrDoS_DNS (0.06), DrDoS_SNMP (0.02)
Syn	Benign (0.45), UDPLag (0.42), Portmap (0.13)
TFTP	DrDoS_MSSQL (0.97), MSSQL (0.02), DrDoS_NTP (0.01)
UDP	DrDoS_UDP (0.93), UDPLag (0.05), MSSQL (0.00)
UDPLag	Syn (0.67), UDP (0.12), DrDoS_UDP (0.11)

H2 est soutenue de façon asymétrique. La majorité des familles inconnues restent détectables ; Syn est le cas dur pour la forêt aléatoire, et la régression logistique est plus robuste. **Q1 :** les familles qui généralisent le mieux sont celles dont le profil d'en-tête, de fenêtre TCP et de débit recouvre celui des familles vues à l'entraînement. Syn, dont le profil TCP recouvre `Benign` et `UDPLag` pour la RF, est l'exception : la matrice confusion-to la prédit majoritairement `Benign` ou `UDPLag`, pas une famille d'attaque unique.

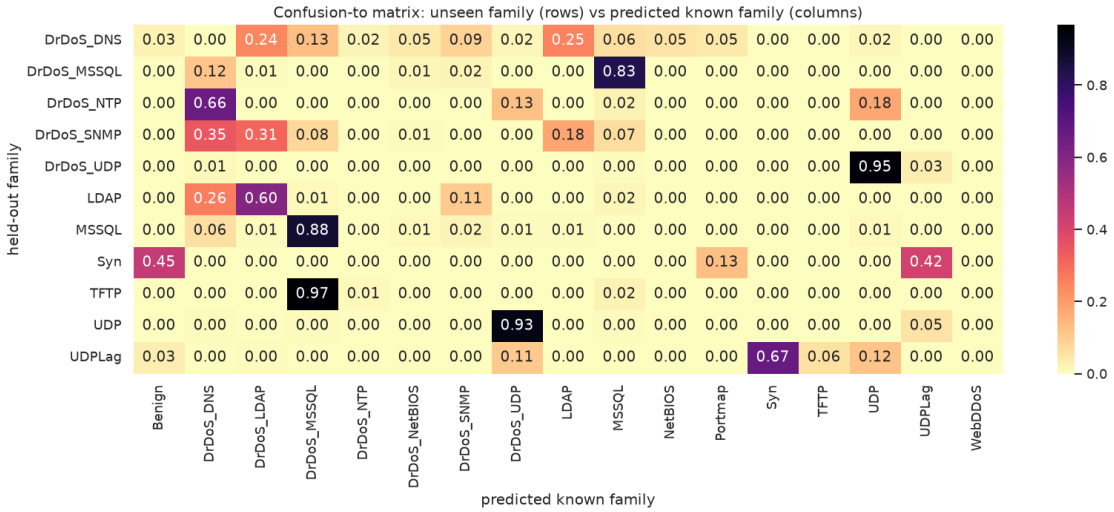


Figure 5: Confusion-to

Interprétation cyber. La matrice de confusion vers les familles connues est plus actionnable qu'un F1 global : elle indique à l'analyste quelle famille connue l'attaque inconnue imite, donc quelle mitigation sera tentée en premier. *Limite.* Le test équilibré 1:1 n'est pas la prévalence de production ; le modèle d'attribution est un diagnostic, pas le détecteur principal.

5.4 Contrôle de robustesse : purger aussi la famille sœur

Dans le jeu de labels fins, `LDAP` et `DrDoS_LDAP` sont deux labels mais un même mécanisme. Le LOADO principal purge seulement le label tenu à l'écart. Le contrôle ci-dessous purge aussi le label frère (`DrDoS_X` quand `X` est tenu à l'écart, et inversement) pour les paires qui ont assez de lignes.

Famille tenue à l'écart	Labels purgés	Macro-F1	Rappel	FPR
LDAP	LDAP, DrDoS_LDAP	0.9985	0.9981	0.0010
DrDoS_LDAP	DrDoS_LDAP, LDAP	0.9980	0.9970	0.0010
MSSQL	MSSQL, DrDoS_MSSQL	0.9993	0.9998	0.0012
DrDoS_MSSQL	DrDoS_MSSQL, MSSQL	0.9993	0.9998	0.0012
UDP	UDP, DrDoS_UDP	0.9996	0.9999	0.0008
DrDoS_UDP	DrDoS_UDP, UDP	0.9996	0.9999	0.0008

Les scores restent élevés : le résultat principal n'est pas uniquement dû à la présence de la famille sœur dans l'entraînement.

6. Expérience 3 — Réduction de features (H3, Q3, Q4)

6.1 Protocole

Cinq variantes : toutes les features, ≈ 75 % (58), ≈ 50 % (39), ≈ 25 % (20) et Top-10. La sélection est ajustée sur le train uniquement avec `mutual_info_classif` ; l'importance de la forêt aléatoire sert de contre-vérification. La latence est la médiane de cinq appels `predict` sur un batch de 1 000 flux, preprocessing inclus.

6.2 Résultats

Variante	Features effectives	Macro-F1 binaire	Macro-F1 13 classes
Toutes	65	0.9989	0.7708
≈ 75 %	58	0.9989	0.7700
≈ 50 %	39	0.9982	0.7658
≈ 25 %	20	0.9963	0.7421
Top-10	10	0.9928	0.6627

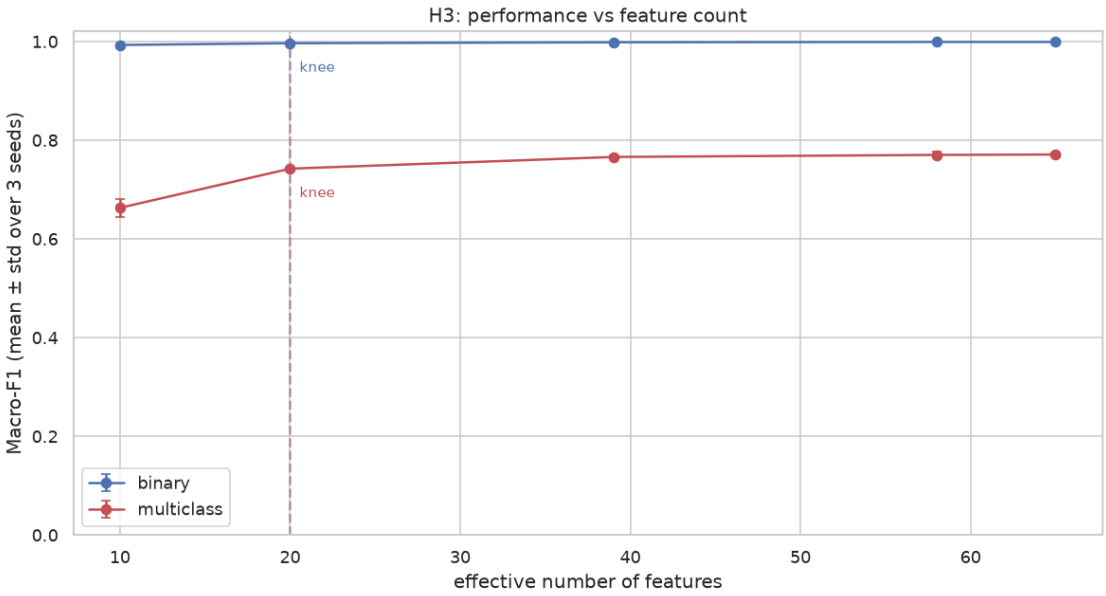


Figure 6: Performance vs nombre de features

Variante	Latence binaire (ms/1k)	Latence 13 classes (ms/1k)
Toutes	46.9942	56.9955
≈ 75 %	47.3736	57.3165
≈ 50 %	46.7709	57.2101
≈ 25 %	46.6769	57.2122
Top-10	46.9381	57.2527

En binaire, la variante Top-10 conserve une Macro-F1 de 0.9928 contre 0.9989 avec toutes les features. La latence reste essentiellement plate : 46.99 ms avec toutes les features contre 46.94 ms en Top-10 par batch de 1 000 flux. Le gain de latence reste faible pour une forêt aléatoire : le coût dominant est la traversée des 300 arbres, pas le nombre de features. En 13 classes, la chute est plus nette : de 0.7708 avec toutes les features à 0.6627 en Top-10. La réduction n'est donc pas gratuite pour les familles rares.

H3 est partiellement soutenue. Q3 : quelques dizaines de features suffisent pour l'essentiel de la performance, mais le nombre exact dépend de la tâche (binaire vs 13 classes) et les familles rares perdent leur rappel en premier. **Q4 :** pour le temps réel, les métriques utiles sont le rappel attaque, le FPR, la latence par batch et le débit ; pas l'accuracy. Un modèle Top-10 est intéressant pour un capteur de bord, mais une forêt de 300 arbres reste probablement trop lente pour une inspection à plein débit.

7. Explicabilité et analyse d'erreurs

L'importance par permutation est calculée sur le **test**, pas sur le train.

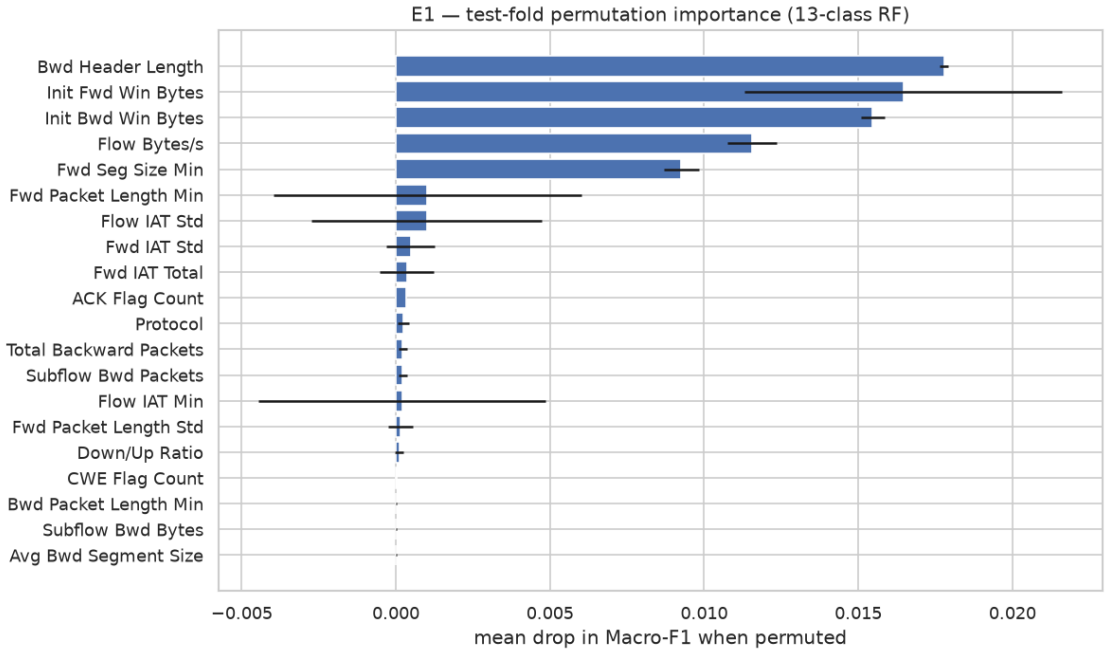


Figure 7: Importance par permutation E1

Feature	Importance moyenne	Écart-type
Bwd Header Length	0.0178	0.0001
Init Fwd Win Bytes	0.0165	0.0052
Init Bwd Win Bytes	0.0155	0.0004
Flow Bytes/s	0.0116	0.0008
Fwd Seg Size Min	0.0093	0.0006
Fwd Packet Length Min	0.0010	0.0050
Flow IAT Std	0.0010	0.0037
Fwd IAT Std	0.0005	0.0008
Fwd IAT Total	0.0004	0.0009
ACK Flag Count	0.0003	0.0000

Les premières features sont des champs d'en-tête TCP et de fenêtre (`Bwd Header Length`, `Init Fwd Win Bytes`, `Init Bwd Win Bytes`) et un débit (`Flow Bytes/s`), pas les tailles moyennes de paquet ni les bulk-rates. Cela explique à la fois le succès binaire et les confusions multiclasses : deux familles qui partagent le même profil d'en-tête et de fenêtre sont difficiles à séparer. Sur les familles quasi parfaitement détectées (`DrDoS_SNMP`, `TFTP`), aucune feature unique ne porte le signal (importance ≈ 0) ; Syn est plus fragile et dépend de `ACK Flag Count`, `Idle Max` et `Init Fwd Win Bytes`.

Les galeries FP/FN (`results/xai_fp_fn_e1.csv`, `results/xai_fp_fn_e2_*.csv`) montrent ce que verrait un analyste : quelques flux Benign avec un profil d'en-tête et de débit agressif, et des attaques Syn à faible débit et faible taux de paquets. Les faux négatifs sur une famille inconnue sont plus dangereux qu'une erreur d'attribution : aucune règle existante ne couvre cette famille.

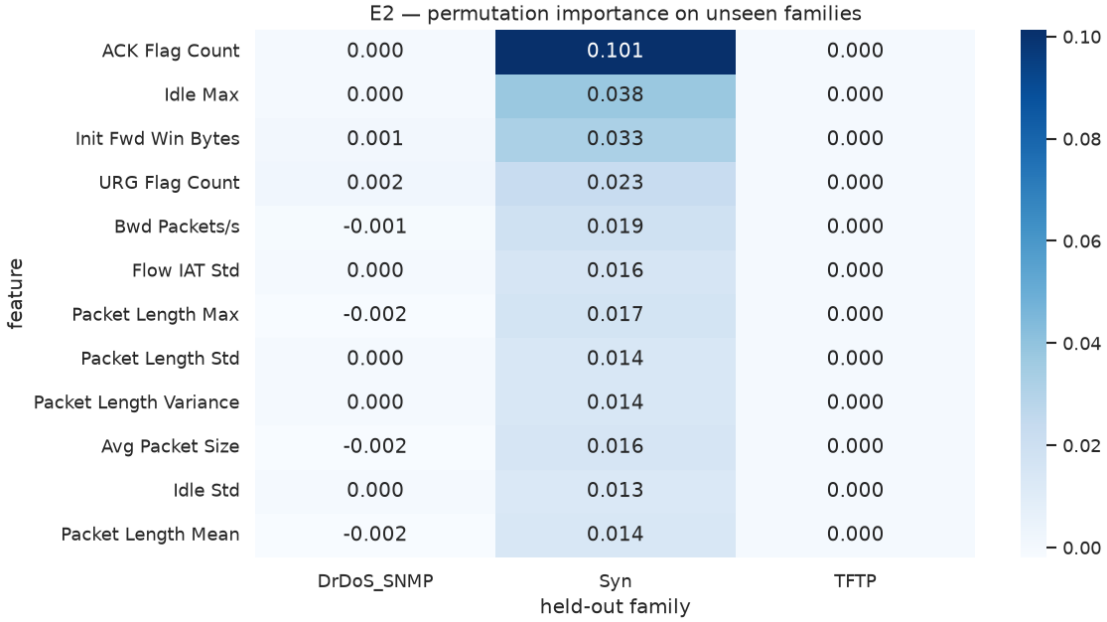


Figure 8: Importance par permutation sur familles inconnues

Limite. L'importance par permutation dépend du modèle et de l'échantillon de test ; les features corrélées se partagent le crédit. Les modèles XAI sont réentraînés avec un cap de 10 000 lignes par classe pour rester exécutables. SHAP n'est pas installé sur Python 3.14 ; aucune conclusion n'en dépend.

8. Discussion, limites et recommandations

8.1 Synthèse

1. Le binaire est facile ; le multiclasses est nettement plus difficile. Un bon score binaire ne prouve pas la compréhension des types d'attaque.
2. La généralisation LOADO reste au-dessus de 0,96 pour dix familles sur onze, mais Syn s'effondre avec la forêt aléatoire alors que la régression logistique la détecte. La dépendance à la famille est donc réelle, concentrée et dépendante du modèle ; la matrice confusion-to explique le cas Syn.
3. Une fraction des features suffit pour l'essentiel de la performance, mais le gain de latence d'une forêt aléatoire est limité ; le choix du classifieur compte autant que le nombre de features.

8.2 Limites

- Dataset de laboratoire 2019 : pas de trafic chiffré, pas de dérive, pas d'adversaire qui adapte son attaque après avoir observé le détecteur.
- Le test E2 est équilibré 1:1 pour rendre FPR et rappel comparables ; la prévalence de production est différente.
- Les familles rares sont exclues des rotations LOADO par manque de support.
- La latence E3 est mesurée sur un laptop en batch, pas sur un équipement réseau.
- Les labels contradictoires pour 5 289 empreintes sont une limite intrinsèque : certains flux sont indistinguables au niveau des features.

8.3 Recommandations

- Utiliser le binaire comme porte de triage, puis un modèle de famille uniquement pour les alertes, avec une classe « inconnue » et une revue humaine.
- Surveiller le rappel par famille et le FPR séparément ; un Macro-F1 global cache les familles rares.
- Pour le temps réel, mesurer la latence de bout en bout et le débit, pas seulement le F1.
- Réentraîner sur des captures récentes et tester la dérive avant tout déploiement.

9. Utilisation de l'IA

Des outils d'IA générative ont été utilisés comme assistance pour structurer le code, proposer des tests de cohérence, déboguer et améliorer la rédaction. Les expériences, les chiffres, les figures et les conclusions proviennent de commandes exécutées dans ce dépôt (`uv run daf-projet ...`). Aucun résultat affirmé sans exécution n'a été conservé. Le groupe reste responsable de l'ensemble du travail et doit pouvoir expliquer chaque choix méthodologique.

10. Conclusion

La réponse à la question centrale est nuancée : un modèle entraîné sur certaines familles DDoS peut reconnaître une famille inconnue, mais cette capacité est asymétrique. Dix familles restent bien détectées ; Syn s'effondre avec la forêt aléatoire alors que la régression logistique résiste. Le binaire reste un bon premier étage ; le multiclasses et l'attribution de famille sont les vrais points durs. La réduction de features est utile, mais elle ne remplace pas un choix de modèle adapté au temps réel. Les résultats les plus importants pour un SOC sont le rappel sur les familles rares, le FPR et la latence, pas l'accuracy globale.

Annexe A — Commandes de reproduction

```
uv sync
uv run daf-projet smoke
uv run daf-projet baseline
uv run daf-projet exp1
uv run daf-projet exp2
uv run daf-projet exp3
uv run daf-projet sibling
uv run jupyter nbconvert --to notebook --execute --inplace notebooks/*.ipynb
uv run python scripts/validate_results.py
```

Annexe B — Fichiers de résultats

- `results/baseline_binary.csv`, `results/exp1.csv`
- `results/exp1_per_class.csv`, `results/exp2.csv`
- `results/exp2_confusion_to.csv`, `results/exp2_purge_audit.csv`
- `results/exp2_sibling_check.csv`, `results/exp3.csv`
- `results/exp3_feature_ranking.csv`, `results/xai_permutation_e1.csv`
- `results/xai_permutation_e2.csv`